Spec2Testbench: Evidence-Grounded Analog Compliance Verification Beyond Simulability

Christian-Marie Moanda Ndeko Mosengo
Independent Researcher
Email: christianmoanda@yahoo.fr

July 15, 2026

Abstract

Successful SPICE simulation establishes that a circuit can be executed, but it does not establish that the measured behavior satisfies a specification. This paper presents Spec2Testbench, a specification-aware verification framework that translates structured requirements into executable ngspice testbenches, extracts traceable measurements through native result backends, and classifies outcomes with an explicit distinction between simulability, compliance, and scientific eligibility. In the canonical nominal benchmark-aligned campaign, 28 circuits were executed in `REAL` mode, 28 completed successfully, 28 remained scientifically eligible, 27 were classified as `SIMULABLE_COMPLIANT`, and one nominal case, `p04_amplifier`, was classified as `SIMULABLE_NONCOMPLIANT` because the extracted `dc_gain_db` value was -160.0000000868589 dB against a requirement of at least 0.0 dB. These results show that Spec2Testbench separates successful execution from specification satisfaction. Secondary evidence confirms operation of both `NGSPICE_MEASURE` and `NGSPICE_WRDATA`, as well as reproducible execution with and without PySpice. The current evidence does not support completed LLM ablation, industrial process-voltage-temperature validation, post-layout claims, or expert-agreement claims, and those items are retained as future work or explicit `TBD` placeholders.

1 Introduction

Analog verification requires more than a simulator return code. A specification-aware workflow must preserve the link between a requirement, an analysis setup, an extracted measurement, and a final compliance verdict. This need becomes especially important when large language models (LLMs), automation frameworks, and benchmark suites are used in analog electronic design automation (EDA), because a generated circuit or a successful simulation can still fail the intended specification [11, 12, 6, 30, 10].

Spec2Testbench addresses this problem as a complementary verification layer. It consumes structured YAML (YAML Ain't Markup Language) specifications, generates or assembles ngspice testbenches, extracts metric evidence through deterministic backends, and issues a final status that distinguishes execution success, simulability, compliance, robustness scope, and scientific eligibility. This positioning is intentionally narrower than automatic circuit generation or optimization: AnalogCoder-Pro is treated here as a circuit-generation-and-optimization framework, whereas Spec2Testbench is positioned as an independent and traceable compliance-evidence layer built downstream of circuit creation [12].

The manuscript is evidence-grounded. All quantitative claims are tied to machine-readable artifacts in the repository, and unsupported historical claims are excluded. The contributions demonstrated in the current evidence are therefore limited and specific: a reproducible specification-to-testbench architecture, a canonical nominal 28-circuit benchmark-aligned evaluation that separates simulability from compliance, native ngspice measurement validation through both `.measure` and `wrdata`, and an explicit account of incomplete or contradictory campaign evidence.

2 Related Work

Recent work has expanded the use of foundation models and automation in analog design, verification, and specification handling. AnalogCoder and AnalogCoder-Pro target analog circuit generation and optimization from textual or multimodal goals, while AnalogTester focuses on automatic analog testbench generation [11, 12, 6]. Other contemporary systems explore adjacent automation boundaries, including analog sizing, optimization, agentic orchestration, or broader design support, as illustrated by AutoCkt, BAGnet, AnalogGym, AnalogXpert, LaMAGIC, AnalogGenie, SPICEPilot, White-Box Reasoning, EasySize, and AnalogAgent [22, 9, 14, 28, 4, 8, 26, 5, 27, 1]. In digital verification, AutoSVA and LLM4DV similarly illustrate how language models can help express test intent without replacing the final checking backend [18, 29].

This paper does not frame these systems in binary

opposition. AnalogCoder-Pro already performs structural, operating-point, direct-current sweep, functional, and multimodal checks during generation and optimization [12]. AnalogTester addresses testbench generation directly [6]. Spec2Testbench instead targets an independent compliance-verification layer with explicit provenance, deterministic checking, and a status taxonomy that prevents missing measurements or mock execution from being silently interpreted as nominal success. This emphasis is closer to evidence accounting than to circuit synthesis.

The simulator and extraction layer are grounded in the long SPICE lineage and in modern ngspice tooling [17, 15, 16, 25]. PySpice provides a Python interface to circuit simulation, but in Spec2Testbench it is an optional convenience layer rather than the authoritative source of the scientific verdict [21]. Broader optimization and robustness context can be found in Bayesian optimization literature and recent variation-aware analog studies, but those references are used here only to situate future work, not to justify unexecuted robustness claims [2, 23, 7, 13, 19, 31, 20, 3, 24].

3 Framework and Methods

3.1 Architecture Overview

Spec2Testbench implements a specification-aware verification chain that separates representation, execution, extraction, and decision. The input is a YAML specification parsed into structured entities and then normalized into a finite contract over metrics, units, thresholds, analyses, and optional case-specific metadata. A verification planner transforms this normalized contract into a test intent that binds each requirement to an analysis family, a stimulus pattern, a load condition, one or more observed nodes, and an evidence route. Testbench construction is then performed either deterministically from templates and registry rules or through an optional LLM-assisted formulation stage that proposes testbench text while leaving the evaluation contract unchanged. The generated testbench is executed through a real ngspice backend, after which a result backend selects either native `.measure` extraction or vector-based `wrdata` extraction. The resulting values are passed to a metric extractor, then to the specification checker, then to a status classifier that separates execution status, simulation mode, compliance status, robustness status, and scientific category. Finally, a report and provenance layer records the measured values, units, metric traces, backend metadata, and run context in machine-readable and human-readable artifacts.

The implementation follows a Clean Architecture interpretation only in the limited sense required for scientific reproducibility: parsing, planning, generation, simulation, extraction, checking, classification, and reporting are isolated responsibilities with explicit data handoffs. This separation reduces hidden coupling between the circuit under test, the testbench, the simulator, and the decision logic, and it makes it possible to audit whether a conclusion comes from the specification, from the generated testbench, from ngspice execution, or from the post-processing layer.

3.2 Specification-to-Testbench Translation

A requirement is translated into an executable testbench contract through a deterministic mapping from semantic intent to simulation evidence. For each expected metric, the planner determines the required analysis type, the excitation or bias stimulus, the load condition, the observed node or signal, the extraction mechanism, the declared unit, and the final assertion applied by the checker. In practice, this means that a specification entry is compiled into an analysis request, one or more source directives, optional load elements, an observation target such as `vout` or supply current, a native `.measure` command or a `wrdata` vector export, a metric name understood by the extractor, a unit normalization path, and a threshold comparison such as range, lower bound, or upper bound.

The translation spans several analysis families. Operating-point and DC requirements are used for quantities such as output bias, quiescent current, or other steady-state voltage and current checks. AC requirements are used for gain- and frequency-related evidence such as `dc_gain_db`, cutoff frequency, unity-gain behavior, or phase-related metrics when these are supported by the specification and backend path. Transient requirements generate time-domain stimuli and timing windows for metrics such as propagation delay, settling behavior, slew-related behavior, and startup amplitude. Spectral requirements use waveform-derived frequency-domain evidence, as in the mixer case where the extractor records quantities such as fundamental frequency or distortion-related outputs. Differential requirements are represented at the specification and planner levels as explicit observed relationships rather than as implicit scalar substitutions. Variation-aware checks are represented as separate requirement families or metadata-scoped scenarios rather than as nominal checks with overloaded semantics; in the present canonical evidence they must be described as a framework capability boundary, not as a completed robustness campaign.

To keep the main text compact, operational command-line usage is intentionally omitted here and can be moved to a compact appendix table. The main methodological point is that the same requirement is preserved across the full path from YAML contract to simulator evidence and back to a checked assertion, so the final decision is tied to an identifiable metric request rather than to a generic simulator success code.

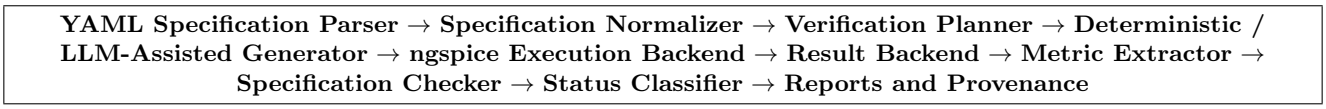

Figure 1: Spec2Testbench architecture overview. The scientific message is separation of responsibilities from specification parsing to evidence reporting.

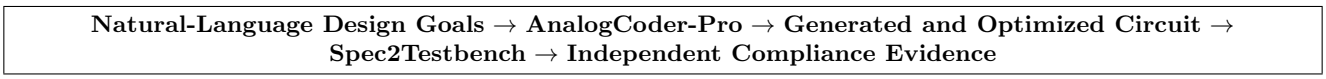

Figure 2: Complementarity with AnalogCoder-Pro. The figure positions Spec2Testbench as a downstream compliance-evidence layer rather than as a replacement for circuit generation or optimization.

3.3 Simulation and Evidence Extraction

All canonical manuscript evidence used here comes from real ngspice execution, not from fabricated or default values. In the nominal paper campaign, all 28 circuits were executed in `REAL` mode with `execution_status = SUCCESS`, and no mock result was retained as scientifically eligible evidence. The execution layer can route extraction through `NGSPICE_MEASURE`, which parses native ngspice `.measure` outputs, or through `NGSPICE_WRDATA`, which exports numeric vectors and recomputes supported metrics from the resulting traces. The canonical validation evidence supports both routes: `NGSPICE_MEASURE` is exercised in the frozen pilot V3 and the native integration reports, whereas `NGSPICE_WRDATA` is validated by two canonical extension cases with independent agreement in both rows of `results/wrdata_independent_comparisons.csv`.

PySpice is optional in this architecture. It is not the authoritative source of the scientific decision, and disabling it does not convert the evaluation into a mock run. Instead, when `SPEC2TESTBENCH_DISABLE_PYSPICE=1`, the framework continues to execute real ngspice runs and uses native extraction and internal parsing paths. The canonical test summary records 66 unique tests passing in both normal mode and PySpice-disabled mode, and the final test artifact explicitly records `mock_results_included = false`. This boundary is important because the paper must distinguish optional convenience tooling from the evidence-producing execution path.

Evidence extraction includes explicit controls against synthetic success. Missing `.measure` values are kept distinct from zero and propagate as `NOT_EVALUATED` rather than as injected numeric defaults. Non-finite values such as NaN or infinity are rejected as non-evaluable measurements. Unit handling is explicit at extraction and checking time so that the final assertion is applied to normalized values with traceable units rather than to raw strings. For oscillatory circuits, frequency acceptance is gated by oscillation validation rather than by the mere presence of an FFT peak. The regression evidence documents this anti-false-PASS behavior directly: missing delay measures now remain null and propagate to `NOT_EVALUATED`, and oscillator frequency is blocked unless a validated oscillation criterion is satisfied.

The provenance layer records the conditions under which each measurement was produced. Case-level reports and provenance structures store identifiers for the specification, circuit, testbench, execution mode, status taxonomy, backend choice, measurement source, and run context. This is also where controlled overrides are made auditable: override application status is preserved in metadata and can prevent a result from being silently interpreted as nominal evidence. Scientifically ineligible mock execution, when explicitly permitted for development purposes, is therefore separable from real evaluation in both status and provenance.

3.4 Deterministic Decision and LLM Boundary

The final scientific decision in Spec2Testbench is deterministic and is made after ngspice execution by the extraction, checking, and classification pipeline. An optional LLM may assist only at the testbench-construction boundary. Concretely, it may propose an analysis form, a stimulus formulation, parts of the testbench text, or measurement directives that instantiate an already fixed requirement. It may therefore influence how the verification question is expressed to the simulator, but not what counts as compliance.

The LLM is not allowed to modify the benchmark circuit during evaluation, the frozen specification, the thresholds, the backend routing, the parser, the checker, or the final decision rule. It also does not decide whether a missing value should pass, fail, or remain unevaluated. These constraints are essential to the anti-false-PASS discipline. The manuscript should therefore present the LLM, when mentioned at all, as an optional front-end assistant for expressing test intent, while the compliance decision remains anchored in deterministic checking over real simulator evidence with explicit statuses for missing measurements, non-simulable cases, and scientifically ineligible mock outputs.

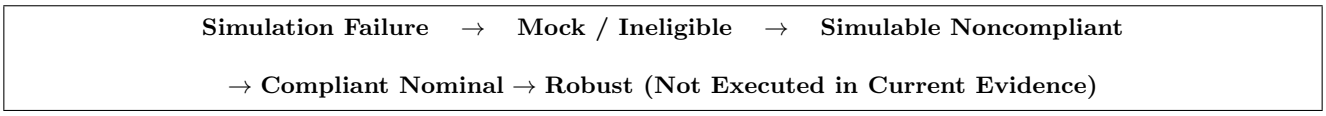

Figure 3: Status space used in the manuscript. Robustness remains a taxonomy boundary rather than a completed result in the present evidence.

4 Experimental Methodology

4.1 Research Questions

The experimental protocol is organized around four research questions. **RQ1** asks whether Spec2Testbench can execute real and reproducible verification runs on the 28-circuit benchmark used in the paper. **RQ2** asks whether the framework distinguishes simulability from specification compliance rather than treating successful ngspice execution as sufficient evidence of correctness. **RQ3** asks whether controlled violations are detected without false accepts or false rejects under the executed mutation campaign. **RQ4** asks what measurable benefit is provided by the LLM-assisted layer relative to the deterministic baseline. No **RQ5** on expert agreement is introduced here because no completed human-validation study was located in the canonical evidence audited for this manuscript.

4.2 Benchmark and Evaluation Scope

The nominal benchmark evaluated in the paper contains 28 local netlists listed in `benchmark/analogcoder_pro/` and referenced by the benchmark manifests and specification files. The repository metadata repeatedly labels this corpus as `AnalogCoder-Pro`, but the manuscript should describe the local evaluation set as *benchmark-aligned* rather than as a formally provenance-verified mirror of an official upstream release, because the present audit did not locate a cryptographic or publication-grade provenance chain linking every local netlist to an official external benchmark package. The benchmark layout documentation in `benchmark/README.md` identifies `benchmark/analogcoder_pro` as the current pedagogical benchmark tier, while the example specification files label the technology as `AnalogCoder-Pro/PySpice generic Level-1 models`. This wording is the safe basis for the manuscript.

At the level of executed nominal evidence, the 28 circuits span 14 circuit families: amplifier, current mirror, comparator, low-pass filter, high-pass filter, band-pass filter, notch filter, opamp, mixer, oscillator, opamp integrator, opamp differentiator, composite arithmetic blocks, and Schmitt trigger. The local netlists are compact rather than industrial-scale: a direct audit of the 28 `.cir` files in `benchmark/analogcoder_pro/` gives a range of 5 to 24 non-empty lines per netlist, with a median of 13 non-empty lines, and a range of 3 to 18 statement-like lines, with a median of 9. These counts are included only as a descriptive complexity proxy for the local benchmark files and should not be overstated as a universal circuit-complexity metric.

The benchmark manifests also document the analyses and expected metrics available to the framework. Across the 28 cases, the configured analyses include DC, AC, transient, and Fourier/spectral paths, depending on circuit family. The configured metric space includes steady-state quantities such as operating point and quiescent current, AC quantities such as `dc_gain_db`, cutoff frequency, and phase margin, transient quantities such as propagation delay, slew-related behavior, settling time, oscillator startup amplitude, and oscillator frequency, and spectral quantities such as THD and fundamental frequency. In the canonical paper campaign, the actually extracted nominal evidence is narrower than the full manifest capability: `results/paper_metric_results.csv` contains executed nominal measurements over operating point, quiescent current, `dc_gain_db`, phase margin, cutoff frequency, propagation delay, oscillator frequency, startup amplitude, slew rate, settling time, THD, and fundamental frequency.

4.3 Execution Environment

The canonical environment record is `results/final_environment.json`. It reports Windows 10 as the operating system, Python 3.12.2, ngspice `ngspice-41 : Circuit level simulation program`, architecture `AMD64`, and Git commit `2373572107fae2e602abf0d0f918be081af09c89`.
The same record reports `mock_allowed = false` for the audited campaign environment and `pyspice_disabled_campaign = true`. The canonical native measurement backends are `NGSPICE_MEASURE` and `NGSPICE_WRDATA`; both are supported by direct executed evidence, but their roles differ across campaigns. `NGSPICE_MEASURE` is the dominant backend in the nominal and frozen-pilot evidence, whereas `NGSPICE_WRDATA` is validated by the two-case extension with independent comparisons.

The paper campaign configuration itself is recorded in `configs/paper_experiment.yaml`. It specifies a reproducible nominal campaign with `paper_eligible_only: true`, `allow_mock: false`, raw-output and provenance preservation enabled, and `llm.enabled: false`. This point is methodologically important: the canonical nominal paper campaign is a deterministic run without an active LLM generation path. As a consequence, no canonical manuscript evidence supports concrete values for LLM temperature, top-p, top-k, candidate count,

or random seeds for the executed paper campaign. If a summary table includes these fields, they should be filled with `TBD -- TO BE FILLED FROM CANONICAL EVIDENCE` or marked as not applicable for the executed nominal run. The same caution applies to framework version strings when they are not explicitly recorded in the canonical environment artifact.

4.4 Experimental Campaigns

The manuscript should distinguish six campaign families and report them separately. The first is the nominal ACP-28 campaign, for which the canonical evidence is `results/paper_campaign_summary.json` and `results/paper_campaign_summary.csv`. This campaign executed 28 benchmark-aligned circuits in `REAL` mode, with `execution_status = SUCCESS` for all cases, `paper_eligible = True` for all cases, 27 `SIMULABLE_COMPLIANT` outcomes, and one `SIMULABLE_NONCOMPLIANT` outcome. The second campaign family is the controlled-violation study. The canonical current summary is `results/final_controlled_summary.json`, which reports 30 generated variants but only 2 effective controlled violations, with one detected effective violation and one false pass. This campaign therefore supports a limited statement about executed controlled violations, but it does not support a blanket claim of zero false accepts or zero false rejects.

The third campaign family is backend validation. This is supported by the frozen pilot V3 and WRDATA extension evidence rather than by the software test suite alone. The canonical frozen pilot summary `results/frozen_pilot_metrics_v3.json` records 16 cases with `TRUE_ACCEPT = 8`, `TRUE_DETECTION = 8`, and no false accepts, false rejects, or unevaluated cases in that frozen dataset. The WRDATA validation evidence is recorded in `results/wrdata_independent_comparisons.csv`, where both independently recomputed vector comparisons agree within tolerance. These backend experiments evaluate evidence extraction behavior and should not be numerically merged with the nominal ACP-28 campaign or with the later expanded controlled-violation campaign.

The fourth, fifth, and sixth campaign families are currently incomplete or absent as scientific evidence. The LLM-versus-deterministic ablation remains partial: `results/final_ablation_summary.json` records `status = PARTIAL`, `A2 = NOT_EXECUTED`, `A4 = NOT_EXECUTED`, and `llm_included = false`. Variation-aware checks and robustness remain unexecuted at the level required for quantitative claims, since `results/final_robustness_metrics.json` records `status = NOT_EXECUTED`. Expert validation likewise remains unavailable in the canonical evidence used here. These three campaign families must therefore be presented as future work, limitations, or `TBD` entries rather than as completed experimental blocks.

4.5 Experimental Metrics

The reported scientific metrics must be tied to the appropriate campaign and must not be confused with software test counts. For the nominal ACP-28 campaign, the relevant metrics are simulation success rate, real simulation rate, paper eligibility rate, nominal compliance rate, simulable-but-noncompliant count, and metric coverage. In the canonical nominal campaign, these quantities are directly available from `results/paper_campaign_summary.json`: simulation success rate is 1.0, real simulation rate is 1.0, paper eligibility rate is 1.0, nominal compliance rate is 0.9642857142857143, simulable-but-noncompliant rate is 0.03571428571428571, and metric extraction success rate is 1.0. Because all 28 nominal cases have nonzero extracted metric counts, the nominal campaign contains no circuits without extracted metrics in the canonical summary.

For the backend-validation and controlled-violation campaigns, the manuscript should use confusion-style metrics that match the actual dataset under discussion. The frozen pilot V3 supports `TRUE_ACCEPT`, `TRUE_DETECTION`, `FALSE_ACCEPT`, `FALSE_REJECT`, and `UNEVALUATED` counts exactly as recorded in `results/frozen_pilot_metrics_v3.json`. The larger controlled-violation campaign supports effective-violation recall, false-pass rate, decision coverage, and unevaluated rate only in its own restricted scope, as recorded in `results/final_controlled_summary.json`. These counts must not be merged into a single unified table unless the table explicitly separates datasets.

No canonical executed evidence currently supports a quantitative estimate of LLM cost, LLM iteration count, or LLM sampling hyperparameters for the paper campaign, because the campaign configuration disables the LLM and the ablation artifact remains partial. If the experimental-methodology section includes a compact campaign table, the LLM-specific fields should therefore be marked `TBD -- TO BE FILLED FROM CANONICAL EVIDENCE` or `not executed`. Likewise, provider unavailability must not be conflated with circuit failure: LLM configuration gaps belong to the method boundary and ablation limitations, whereas circuit execution outcomes belong to the ngspice-based scientific campaigns.

5 Results

5.1 RQ1: Real Execution

Answer to RQ1. Yes for real execution and reproducibility, within the scope of the nominal benchmark-aligned paper campaign. The canonical nominal run executed 28 circuits, all in `REAL` mode, with no mock result

Table 1: Nominal ACP-28 execution summary from the canonical paper campaign (`results/paper_campaign_summary.json`, run id `20260711_094959`).

Metric	Value
Circuits executed	28
REAL-mode runs	28
Mock runs	0
Successful executions	28
Scientifically eligible results	28
Simulation success rate	1.0000
Real simulation rate	1.0000
Paper eligibility rate	1.0000
Metric extraction success rate	1.0000
Analysis families covered in extracted evidence	DC / AC / transient / spectral

Table 2: Compact nominal compliance table from the canonical paper campaign. The full per-circuit table can be moved to an appendix if required.

Circuit	Mode	Execution	Key metric	Threshold	Scientific category
`p01_amplifier–p03_amplifier`	REAL	SUCCESS	extracted	case-specific	SIMULABLE_COMPLIANT
`p04_amplifier`	REAL	SUCCESS	`dc_gain_db = -160.0000000868589 dB`	≥ 0.0 dB	SIMULABLE_NONCOMPLIANT
`p05_amplifier–p28_schmitt` except `p04`	REAL	SUCCESS	extracted	case-specific	SIMULABLE_COMPLIANT

retained in the scientific evidence, 28 successful executions, and 28 scientifically eligible outputs.

The direct evidence comes from the canonical paper campaign `results/paper_campaign_summary.json` and `results/paper_campaign_summary.csv`, run id `20260711_094959`. All 28 rows have `simulation_mode = REAL`, `execution_status = SUCCESS`, and `paper_eligible = True`. The same campaign covers multiple analysis families through the extracted nominal metrics: operating-point and DC quantities such as operating point and quiescent current, AC quantities such as `dc_gain_db` and cutoff frequency, transient quantities such as propagation delay, slew rate, settling time, startup amplitude, and oscillator frequency, and spectral quantities such as THD and fundamental frequency. These results establish that Spec2Testbench can produce real and traceable verification evidence across the benchmark-aligned corpus.

The interpretation is limited to execution and evidence production. Successful real execution does not by itself establish that a circuit is specification-compliant, and the next subsection shows why this distinction is necessary.

5.2 RQ2: Simulability Versus Compliance

Answer to RQ2. Yes. The canonical nominal campaign shows that Spec2Testbench distinguishes simulability from compliance: 28 circuits are simulable in the nominal campaign, but only 27 are compliant and one is noncompliant.

The nominal result is directly visible in `results/paper_campaign_summary.csv` and `results/simulability_vs_compliance.csv`. The campaign contains 27 `SIMULABLE_COMPLIANT` cases and one `SIMULABLE_NONCOMPLIANT` case. The unique noncompliant nominal case is `p04_amplifier`. Its per-case artifact `artifacts/paper_campaign/20260711_094959/p04_amplifier/report.json` shows `execution_status = SUCCESS`, `simulation_mode = REAL`, and `scientific_category = SIMULABLE_NONCOMPLIANT`. The metric record in `results/paper_metric_results.csv` shows that `dc_gain_db` was extracted successfully with measured value -160.0000000868589 dB against the requirement ≥ 0.0 dB, yielding a noncompliant verdict.

This case provides the direct counterexample needed by the paper:

$$\text{SIMULABLE} \neq \text{COMPLIANT}$$

Spec2Testbench therefore does not collapse successful ngspice termination into specification satisfaction. The result is scientifically useful precisely because it preserves the difference between executable evidence and compliant behavior.

5.3 RQ3: Controlled Violations

Answer to RQ3. The canonical evidence does not support a claim of perfect controlled-violation detection across all executed datasets. Instead, it supports two distinct statements: the frozen pilot V3 achieved perfect separation within its limited 16-case scope, whereas the later expanded controlled-violation campaign retained one false accept among two effective violations.

The retained frozen pilot evidence is `results/frozen_pilot_metrics_v3.json`. This dataset records 16 cases with `TRUE_ACCEPT = 8`, `TRUE_DETECTION = 8`, `FALSE_ACCEPT = 0`, `FALSE_REJECT = 0`, and

Table 3: Controlled-violation results must be separated by dataset. The frozen pilot V3 is retained as the canonical small-scope confusion matrix, while the later expanded campaign is reported as a contradictory sensitivity result rather than silently merged.

Dataset	Date / run id	True accept	True detection	False accept	False reject	Unevaluated
Frozen pilot V3	canonical V3 file	8	8	0	0	0
Expanded controlled campaign	20260712_195226	TBD	1 effective detection	1 effective false pass	0 reported	0 among effective cases

Table 4: Expanded controlled-violation campaign summary restricted to the executed effective-violation population.

Metric	Value
Generated variants	30
Effective controlled violations	2
Ineffective mutations	28
Detected effective violations	1
False accepts among effective violations	1
False rejects reported	0
Unevaluated effective violations	0
Violation detection recall	0.5000
False-pass rate	0.5000

`UNEVALUATED = 0`. It also includes two WRDATA extension cases and is the canonical source for the small-scope backend-validated pilot. The later expanded campaign is `results/final_controlled_summary.json`, dated by run id `20260712_195226`. That campaign generated 30 variants, but only 2 were effective controlled violations and 28 were ineffective mutations. Within those 2 effective violations, one was detected and one was missed, yielding one false pass and zero unevaluated effective cases. This later campaign is therefore the canonical source for the broader sensitivity analysis, not for a perfect-detection claim.

The manuscript should not silently merge these datasets. The frozen pilot V3 is retained as the canonical small-scope campaign for a clean confusion matrix because it is internally consistent, backend-validated, and machine-readable. The contradictory later campaign must be reported explicitly as a limitation or sensitivity result: it shows that the broader mutation population was far harder than the frozen pilot and that perfect controlled-violation detection did not generalize to the expanded effective set.

The interpretation is therefore two-level. In the frozen pilot V3, Spec2Testbench achieved perfect discrimination on the retained 16-case pilot. In the later expanded mutation campaign, the framework still separated effective from ineffective mutation accounting, but one effective violation remained falsely accepted. This is a limitation of the broader controlled-violation evidence, not a reason to overwrite the frozen pilot counts.

5.4 RQ4: Baseline Versus LLM

Answer to RQ4. No quantitative answer can be given from the current canonical evidence because the required ablation results are not yet available.

The canonical ablation artifact `results/final_ablation_summary.json` records `status = PARTIAL`, `A2 = NOT_EXECUTED`, `A4 = NOT_EXECUTED`, and `llm_included = false`. The canonical nominal paper campaign configuration in `configs/paper_experiment.yaml` also sets `llm.enabled: false`. As a result, the manuscript cannot report valid generation rate, simulation rate, metric coverage, correct verdict rate, false accept rate, cost, or run-to-run variability for an executed LLM-versus-deterministic comparison. Table 5 is therefore intentionally left as a pending-results table rather than a quantitative result table.

The interpretation is straightforward: the LLM layer remains a planned experimental factor rather than a completed results block in the current manuscript evidence.

5.5 Secondary Software Results

Answer to the secondary software question. The software evidence supports backend functionality and reproducible execution infrastructure, but it is not used here as direct proof of analog specification compliance.

The canonical software-test artifact `results/final_test_results.json` records 66 unique tests with 66 passes, 0 failures, 0 skips, and 1 warning in both normal mode and PySpice-disabled mode. The backend evidence separately confirms that `NGSPICE_MEASURE` and `NGSPICE_WRDATA` both operate on canonical evidence, and `results/wrdata_independent_comparisons.csv` shows agreement in both WRDATA validation rows. These software and backend results establish that the execution and extraction machinery functions with and without PySpice, but they are presented here only as supporting infrastructure evidence. The analog-compliance claims in this section remain tied to the campaign artifacts and per-case metric records rather than to the software test suite.

6 Discussion

The results support four bounded conclusions. First, Spec2Testbench can execute real ngspice verification runs

Table 5: Planned LLM Ablation — Results Pending. No canonical quantitative LLM ablation results are available in the current manuscript evidence.

Configuration	Valid generation rate	Simulation rate	Metric coverage	Correct verdict rate	False accept rate	Cost / variability
Deterministic baseline	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE
LLM-assisted generation	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE	TBD – TO BE FILLED FROM CANONICAL EVIDENCE

Table 6: Secondary software results reported separately from analog-compliance evidence.

Software / backend item	Canonical value
Pytest unique tests	66
Normal-mode passes	66
PySpice-disabled passes	66
WRDATA independent comparisons	2 / 2 agreement
Mock results included in canonical test artifact	0

Circuit: `p04_amplifier`
Analysis: AC
Metric: `dc_gain_db`
Threshold: ≥ 0.0 dB
Measured value: -160.0000000868589 dB
Execution: SUCCESS / REAL
Verdict: `SIMULABLE_NONCOMPLIANT`

Figure 4: The canonical nominal counterexample showing that a successfully simulated circuit can still be noncompliant.

Frozen pilot V3:
TRUE_ACCEPT = 8
TRUE_DETECTION = 8
FALSE_ACCEPT = 0
FALSE_REJECT = 0
Expanded campaign:
Effective violations = 2
Detected = 1
False accept = 1
Unevaluated = 0

Figure 5: Controlled-violation matrix summary. The two datasets are displayed separately to avoid silently merging incompatible populations.

Example specification-to-evidence path
YAML requirement → generated testbench
→ ngspice waveform or measure output
→ extracted metric with unit
→ assertion against threshold
→ final verdict with provenance

Figure 6: Example of the specification-to-evidence translation chain represented in the manuscript.

across the benchmark-aligned ACP-28 corpus with no retained mock evidence in the canonical nominal campaign. Second, the framework distinguishes simulability from compliance, as demonstrated by the unique nominal noncompliant case `p04_amplifier`. Third, native evidence extraction is supported through both `NGSPICE_MEASURE` and `NGSPICE_WRDATA`, including operation when PySpice is disabled. Fourth, controlled-violation evidence must be described with dataset-specific scope: the frozen pilot V3 supports a clean 16-case confusion matrix, whereas the later expanded campaign introduces one false accept among two effective violations.

The limitations are equally important. The benchmark uses compact pedagogical netlists with generic Level-1 device models rather than industrial post-layout circuits. The local corpus is therefore described as benchmark-aligned with AnalogCoder-Pro rather than as a provenance-verified official upstream mirror. Simplified variation-aware checks and the presence of temperature or supply sweeps in specifications must not be presented as full industrial process-voltage-temperature (PVT) validation. The historical Compliance Score is not used as primary evidence because the current manuscript relies instead on explicit metric values, thresholds, and status classes. Likewise, the LLM layer is deliberately not oversold: the canonical executed paper campaign disables it, and the current ablation evidence remains partial.

These constraints shape the scientific claim of the paper. The present evidence supports a reproducible and traceable compliance-verification layer for academic benchmark circuits, not a universal sign-off methodology. The most credible next steps are a fully executed deterministic-versus-LLM ablation, a rigorously calibrated larger controlled-violation study, and a separate robustness campaign with clearly delimited variation assumptions.

7 Conclusion

Spec2Testbench contributes an evidence-centered analog verification workflow that preserves the distinction between simulation success and specification compliance. In the current canonical evidence, the framework executes 28 real nominal runs, classifies 27 as compliant and one as noncompliant, validates both native measurement backends, and records clear boundaries around incomplete ablation and robustness evidence. The manuscript therefore advances a precise claim: Spec2Testbench is a complementary and traceable compliance-verification layer for benchmark-aligned analog circuits, and its strongest current result is the deterministic production of auditable evidence rather than an inflated claim of universal correctness.

References

[1] Zhixuan Bao, Zhuoyi Lin, Jiageng Wang, Jinhai Hu, Yuan Gao, Yaoxin Wu, Xiaoli Li, and Xun Xu. Analogagent: Self-improving analog circuit design automation with llm agents, 2026.

[2] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning, 2010.

[3] Paul Brokaw. A simple three-terminal ic bandgap reference. *IEEE Journal of Solid-State Circuits*, December 1974.

[4] Chen-Chia Chang, Yikang Shen, Shaoze Fan, Jing Li, Shun Zhang, Ningyuan Cao, Yiran Chen, and Xin Zhang. Lamagic: Language-model-based topology generation for analog integrated circuits, 2024.

[5] Jianqiu Chen, Siqi Li, and Xu He. White-box reasoning: Synergizing llm strategy and gm/id data for automated analog circuit design, 2025.

[6] Weiyu Chen, Chengjie Liu, Wenhao Huang, Jinyang Lyu, Mingqian Yang, Yuan Du, Li Du, and Jun Yang. Analogtester: A large language model-based framework for automatic testbench generation in analog circuit design, 2025.

[7] Peter I. Frazier. A tutorial on bayesian optimization, 2018.

[8] Jian Gao, Weidong Cao, Junyi Yang, and Xuan Zhang. Analoggenie: A generative engine for automatic discovery of analog circuit topologies, 2025.

[9] Kourosh Hakhamaneshi, Nick Werblun, Pieter Abbeel, and Vladimir Stojanovic. Bagnet: Berkeley analog generator with layout optimizer boosted with deep neural networks, 2019.

[10] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. ChatEDA: A large language model powered autonomous agent for EDA, 2023.

[11] Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z. Pan, and Ping Luo. Analogcoder: Analog circuit design via training-free code generation, 2024.

[12] Yao Lai, Souradip Poddar, Sungyoung Lee, Guojin Chen, Mengkang Hu, Bei Yu, Ping Luo, and David Z. Pan. Analogcoder-pro: Unifying analog circuit generation and optimization via multi-modal llms, 2025.

[13] Martin Lefebvre and David Bol. A family of current references based on 2t voltage references: Demonstration in 0.18-μm with 0.1-na ptat and 1.1-μa cwt 38-ppm/°c designs, 2022.

[14] Jintao Li, Haochang Zhi, Ruiyu Lyu, Wangzhen Li, Zhaori Bi, Keren Zhu, Yanhan Zeng, Weiwei Shan, Changhao Yan, Fan Yang, Yun Li, and Xuan Zeng. Analoggym: An open and practical testing suite for analog circuit synthesis, 2024.

[15] L. W. Nagel and D. O. Pederson. SPICE (Simulation Program with Integrated Circuit Emphasis). Technical report, University of California, Berkeley, 1973. Memorandum No. ERL-M382.

[16] Laurence W. Nagel. SPICE2: A computer program to simulate semiconductor circuits. Technical report, University of California, Berkeley, 1975. Memorandum No. ERL-M520.

[17] Laurence W. Nagel and Ronald A. Rohrer. Computer analysis of nonlinear circuits, excluding radiation (CANCER). *IEEE Journal of Solid-State Circuits*, 1971.

[18] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. AutoSVA: Democratizing formal verification of rtl module interactions, 2021.

[19] Shubham Patil, Anand Sharma, Gaurav R, Abhishek Kadam, Ajay Kumar Singh, Sandip Lashkare, Nihar Ranjan Mohapatra, and Udayan Ganguly. Process voltage temperature variability estimation of tunneling current for band-to-band-tunneling based neuron, 2023.

[20] Ria Rashid, Gopavaram Raghunath, Vasant Badugu, and Nandakumar Nambath. Performance evaluation of evolutionary algorithms for analog integrated circuit design optimisation, 2023.

[21] Fabrice Salvaire and contributors. Pyspice: Simulate electronic circuit using python and the ngspice / xyce simulators, 2026. Official project repository accessed 2026-07-15.

[22] Keertana Settaluri, Ameer Haj-Ali, Qijing Huang, Kourosh Hakhamaneshi, and Borivoje Nikolic. Autockt: Deep reinforcement learning of analog circuit designs, 2020.

[23] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms, 2012.

[24] Y. P. Tsividis. Accurate analysis of temperature effects in ic-vbe characteristics with application to bandgap reference sources. *IEEE Journal of Solid-State Circuits*, 15(6):1076–1084, December 1980.

[25] Holger Vogt, Giles Atkinson, Dietmar Warning, and Paolo Nenzi. *Ngspice User's Manual, Version 46*, March 2026.

[26] Deepak Vungarala, Sakila Alam, Arnob Ghosh, and Shaahin Angizi. SPICEPilot: Navigating SPICE code generation and simulation with ai guidance, 2024.

[27] Xinyue Wu, Fan Hu, Shaik Jani Babu, Yi Zhao, and Xinfei Guo. Easysize: Elastic analog circuit sizing via llm-guided heuristic search, 2025.

[28] Haoyi Zhang, Shizhao Sun, Yibo Lin, Runsheng Wang, and Jiang Bian. Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models, 2024.

[29] Zixi Zhang, Greg Chadwick, Hugo McNally, Yiren Zhao, and Robert Mullins. LLM4DV: Using large language models for hardware test stimuli generation, 2023.

[30] Ruizhe Zhong, Xingbo Du, Shixiong Kai, Zhentao Tang, Siyuan Xu, Hui-Ling Zhen, Jianye Hao, Qiang Xu, Mingxuan Yuan, and Junchi Yan. LLM4EDA: Emerging progress in large language models for electronic design automation, 2023.

[31] Junzhuo Zhou, Ziwen Wang, Haoxuan Xia, Yuxin Yan, Chengyu Zhu, Ting-Jung Lin, Wei Xing, and Lei He. Setupkit: Efficient multi-corner setup/hold time characterization using bias-enhanced interpolation and active learning, 2025.